

Đánh giá kiến trúc MLOps và lộ trình Degradation-Aware CT

Hiện trạng, khoảng trống và lộ trình Degradation-Aware Continuous Training

August 28, 2026

Abstract

Báo cáo này tiến hành đánh giá toàn diện repository MLOps PaaS dựa trên khung kiến trúc tham chiếu chuẩn mực của Amou Najafabadi *et al.* [1]—một nghiên cứu tổng hợp (systematic mapping study) từ 93 công trình khoa học, bao quát 39 thành phần kiến trúc, 56 bước quy trình và 15 vai trò stakeholder. Kết quả phân tích mã nguồn thực tế cho thấy hệ thống đã thiết lập nền tảng vững chắc cho một nền tảng vận hành mô hình (MLOps PaaS): control plane phân tách đa tenant, registry quản lý phiên bản bất biến, hạ tầng huấn luyện đa backend (Docker/Argo/Kubeflow), dịch vụ đóng gói tự động (model-packager), serving gateway và luồng thu thập production log phục vụ giám sát data drift.

Tuy nhiên, đối chiếu theo phân loại của nghiên cứu tham chiếu, hệ thống hiện dừng lại ở biến thể **V1 (Inference Service + Manual Retraining)**, chưa hình thành vòng lặp **Continuous Training (CT - biến thể V3)** khép kín. Các khoảng trống kiến trúc trọng yếu bao gồm: thiếu cơ sở dữ liệu phản hồi nhận thực tế (Feedback Database), chưa có cơ chế đánh giá hiệu năng suy giảm (performance degradation), thiếu công cụ so sánh tự động candidate–champion và chưa hỗ trợ điều phối lưu lượng shadow/canary. Trên cơ sở đó, báo cáo đề xuất hướng nghiên cứu và luận đề **“Degradation-Aware Adaptive Model Maintenance”** (Bảo trì mô hình thích ứng nhận thức suy giảm), phân tách tường minh ba lớp: phát hiện (detection), chẩn đoán nguyên nhân (diagnosis) và ra quyết định hành động (action selection). Trong kiến trúc này, tái huấn luyện (retraining) là một hành động có điều kiện kiểm chứng nghiêm ngặt thay vì phản xạ mặc định sau mỗi cảnh báo drift.

1 Phạm vi và phương pháp đánh giá

1.1 Khung kiến trúc tham chiếu

Nghiên cứu tham chiếu của Amou Najafabadi *et al.* [1] mô tả hệ thống MLOps thông qua hai chiều cạnh cấu trúc cốt lõi:

- **Cấu trúc tĩnh (Structure):** Gồm 39 thành phần chức năng được phân bổ trong 6 nhóm: (1) Data Curation, (2) Storage & Versioning, (3) ML Training, (4) CI/CD, (5) Inference, và (6) Infrastructure & Supporting Services.
- **Quy trình động (Process):** Gồm 56 bước quy trình bao quát trọn vẹn vòng đời từ thu thập dữ liệu, huấn luyện, kiểm thử, triển khai đến giám sát và bảo trì trong môi trường sản xuất.
- **Không gian biến thể (Variants):** Phân định 8 biến thể kiến trúc (V1–V8). Trong đó, nhóm cốt lõi V1–V4 áp dụng cho mô hình huấn luyện từ đầu (train from scratch), phân biệt bởi hình thức phục vụ dự đoán (Inference Service độc lập vs. Inference Engine Pool dùng chung) và phương thức tái huấn luyện (Manual vs. Continuous Training). Các biến thể V5–V8 tương ứng áp dụng cho mô hình nạp sẵn (pre-trained/fine-tuned).

Khung tham chiếu trên được sử dụng như một chuẩn mực đánh giá cấu trúc và phân tích khoảng trống (gap analysis), giúp định hình hướng phát triển học thuật thay vì xem xét như một danh mục kiểm tra cứng nhắc.

1.2 Phương pháp thẩm định mã nguồn

Đánh giá được thực hiện trực tiếp trên mã nguồn, schema cơ sở dữ liệu, đặc tả hạ tầng Kubernetes/Docker và các bộ kiểm thử tự động của repository. Các luồng nghiệp vụ chính được rà soát bao gồm: (1) upload/build artifacts, (2) training execution & tracking, (3) model serving & event stream, (4) model registry & deployment, và (5) drift monitoring workflow. Một thành phần được ghi nhận là “Thiếu” thể hiện rằng nền tảng chưa cung cấp contract hoặc module cài đặt sẵn trong hệ thống quản trị, không hàm ý người dùng bị giới hạn khi tự định nghĩa logic trong mã nguồn mô hình riêng.

2 Kết luận điều hành (Executive Summary)

Tổng thể mã nguồn xác nhận repository là một **nền tảng MLOps PaaS mạnh mẽ ở khâu vận hành và triển khai mô hình (Model Operations)**, nhưng chưa hoàn thiện năng lực **tự thích ứng và bảo trì mô hình khép kín (Closed-Loop Continuous Training)**. Điểm mạnh nổi bật nằm ở kiến trúc phân tầng rõ ràng (Control Plane vs. Data Plane vs. Execution Plane), tính bất biến của phiên bản mô hình, tính đa dạng của backend huấn luyện và kênh thu thập sự kiện suy luận. Ngược lại, điểm nghẽn lớn nhất nằm ở sự thiếu vắng vòng lặp phản hồi nhân thực tế, thiếu cơ chế giám sát chất lượng/hiệu năng suy giảm và chưa có trí tuệ ra quyết định (decision intelligence) giữa khâu cảnh báo và kích hoạt tái huấn luyện.

Table 1: Đánh giá tổng hợp năng lực hệ thống theo khung kiến trúc tham chiếu.

Trục kiến trúc	Mức độ	Nhận định tổng quan
Nền tảng & Phân tầng	7/10	Phân tách rõ ràng giữa Control, Data và Execution Plane; phân quyền tenant chuẩn mực.
Lưu trữ & Quản lý phiên bản	7/10	Quản lý artifact, digest image và snapshot huấn luyện bất biến; thiếu Feedback DB.
Huấn luyện & CI/CD	7/10	Hỗ trợ đa backend (Docker/Argo/Kubeflow), đóng gói tự động; thiếu Evaluation Gate tự động.
Suy luận & Quan sát	6/10	Gateway serving tốt, thu thập event stream ổn định; chưa đo lường performance thực tế.
Vòng đời bảo trì khép kín	4/10	Tự động kích hoạt DriftRun khi đủ mẫu; chưa có cơ chế chặn đoán nguyên nhân và CT policy.
Quản trị & Stakeholder	4/10	Có xác thực JWT/API-Key và audit log; chưa có luồng phê duyệt (approval) cho bảo trì mô hình.

Xác định biến thể kiến trúc hiện tại: Đối chiếu với phân loại của Amou Najafabadi *et al.* [1], hệ thống hiện đáp ứng trọn vẹn đặc trưng của **Biến thể V1 (Inference Service + Manual Retraining)**: dịch vụ model-packager đóng gói mô hình thành container độc lập, task deployment đẩy lên hạ tầng production, và model-server định tuyến phục vụ dự đoán. Việc tái huấn luyện hoàn toàn phụ thuộc vào thao tác thủ công của kỹ sư ML. Mục tiêu của đề tài là nghiên cứu và nâng cấp hệ thống từ biến thể V1 lên **Biến thể V3 (Continuous Training Pipeline)** có tích hợp cơ chế nhận thức suy giảm thích ứng.

3 Đối chiếu cấu trúc kiến trúc — Mapping 39 thành phần

Phần này ánh xạ chi tiết 39 thành phần chức năng của khung kiến trúc tham chiếu [1] với hiện trạng mã nguồn thực tế của repository, chia theo 6 nhóm chức năng. Sơ đồ Hình 1 mô tả mối quan hệ tương tác giữa 6 nhóm nền tảng và làm nổi bật vòng lặp ra quyết định (Decision Loop) đề xuất bổ sung.

3.1 Data Curation

Table 2: Ánh xạ nhóm Data Curation.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Data Collector	Một phần	Workspace assets hỗ trợ upload mã/dữ liệu; TrainingJob nhận data_snapshot_uri.	Chưa có đường ống thu nạp dữ liệu tự động (automated data ingestion) từ nguồn bên ngoài.
Data Source	Một phần	Hỗ trợ nạp dataset qua workspace; asset tham chiếu (reference asset) cho drift.	Nền tảng chưa quản trị kết nối trực tiếp tới các nguồn dữ liệu ngoại vi (APIs, Streaming, DBs).
Data Preprocessor	Thiếu	Tiền xử lý dữ liệu được thực thi trực tiếp trong mã người dùng tại entry point.	Chưa có pipeline tiền xử lý và chuẩn hóa dữ liệu được quản lý ở cấp độ nền tảng (platform-managed).

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Data Quality Checker	Thiếu	Chưa có module hiện thực sẵn trong nền tảng.	Khoảng trống trọng yếu; cần thiết để ngăn chặn việc tái huấn luyện trên dữ liệu sai hỏng/lỗi schema.
Data Labeler	Thiếu	Chưa có luồng gán nhãn hoặc tích hợp hệ thống annotation.	Cần thiết cho vòng lặp tiếp nhận nhãn trễ (delayed-label feedback loop).

3.2 Storage & Versioning

Table 3: Ánh xạ nhóm Storage & Versioning.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Data Catalog	Thiếu	Chưa có metadata catalog quản lý danh mục dataset tập trung.	Cần thiết nhằm truy xuất toàn diện nguồn gốc dữ liệu phục vụ mô hình (data → model lineage).
Dataset Repository	Một phần	S3 lưu trữ các snapshot dữ liệu huấn luyện (data_snapshot_uri); workspace assets.	Dữ liệu chưa được mô hình hóa thành thực thể phiên bản độc lập; lưu trữ dạng snapshot tĩnh.
Raw Data Store	Một phần	S3 lưu trữ raw/reference assets; production logs lưu trong PostgreSQL.	Chưa phân tách triệt để giữa tầng lưu trữ dữ liệu thô (raw), sơ chế (processed) và dữ liệu chuẩn (curated).
Feature Store	Thiếu	Chưa có module hiện thực trong hệ thống.	Các đặc trưng hiện được tính toán trực tiếp trong mã nguồn người dùng.
Code Repository	Có	Quản lý code_snapshot_uri bất biến trong TrainingJob; liên kết Git repo.	Đã đáp ứng trọn vẹn yêu cầu quản lý phiên bản mã nguồn huấn luyện.
Model Repository	Có	ModelVersion + ModelArtifact + ModelMetric trong ứng dụng registry; phiên bản bất biến, metrics/params summary, artifacts.	Đạt chuẩn mực cao; đây là năng lực nền tảng cốt lõi của hệ thống.
Artifact Repository	Có	S3 + thực thể ModelArtifact lưu trữ container image và artifacts theo phiên bản; checksum cố định.	Đã đáp ứng tốt cho biến thể V1; cần mở rộng liên kết với công thẩm định candidate.
ML Metadata Repository	Có	Tracking TrainingJob (JSON), TrainingOutput, TrainingJobEvent, RegistryEvent; tích hợp MLflow.	Đã hoàn thiện ở mức nền tảng; cần bổ sung thêm environment lineage và configuration lineage.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Feedback Database	Thiếu	Schema PostgreSQL có các trường runtime nhưng chưa có Feedback API/Store cho ground truth.	Khoảng trống then chốt —cần xây dựng bảng Feedback liên kết trực tiếp với prediction_id.
Predictions Storage	Có	Bảng paas_production_logs lưu trữ log suy luận phân vùng theo tenant/project/version.	Đã có lưu trữ, nhưng cần bổ sung các trường runtime (confidence, latency, status) vào event stream.

3.3 ML Training

Table 4: Ảnh xạ nhóm ML Training.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Feature Engineering Pipeline	Một phần	Biến đổi đặc trưng nằm trong mã người dùng; chưa có pipeline quản lý tập trung.	Cần chuẩn hóa biến đổi và phiên bản hóa đặc trưng khi logic feature ảnh hưởng tới CT.
ML Experiment Pipeline	Có	TrainingJob hỗ trợ entry point, snapshot mã/dữ liệu, tích hợp MLflow; Docker hoặc Argo/Kubeflow.	Nền tảng thực thi thử nghiệm hoạt động tin cậy và linh hoạt.
Continuous Training Pipeline	Thiếu	Chưa có luồng tự động hóa tái huấn luyện khép kín.	Trọng tâm nghiên cứu cốt lõi của đề tài.
Model Evaluator	Một phần	Trích xuất metrics qua MLflow/TrainingOutput; chưa có cổng thẩm định đối kháng tự động.	Cần xây dựng thực thể EvaluationGate phục vụ so sánh Champion–Candidate.
Fine-Tuning Module	Thiếu	Chưa hỗ trợ quy trình tinh chỉnh mô hình nạp sẵn chuyên biệt.	Thành phần mở rộng cho các biến thể V5–V8 trong tương lai.

3.4 CI/CD

Table 5: Ảnh xạ nhóm CI/CD.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
ML Component Builder	Có	Dịch vụ model-packager đóng gói container image từ training artifacts; theo dõi trạng thái, digest.	Đã hoàn thiện vững chắc cho quy trình đóng gói tự động.
ML Component Deployer	Có	Thực thể Deployment, Endpoint, Kustomize/GitOps manifests; Celery task orchestration.	Đã đáp ứng tốt việc triển khai containerized component lên hạ tầng.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
ML Model Deployer	Thiếu	Chưa hỗ trợ nạp mô hình động vào một Inference Engine Pool dùng chung.	Chỉ áp dụng khi mở rộng sang biến thể V2/V4.
ML Pipeline Builder	Một phần	Cấu hình Argo Workflows / Kubeflow pipeline manifests có sẵn trong hệ thống.	Đóng gói pipeline huấn luyện chuẩn hóa qua Argo templates, không tự xây dựng orchestrator mới.
ML Pipeline Deployer	Một phần	Kích hoạt qua Celery tasks và Kubernetes CRDs.	Kết nối RetrainingRequest với Argo Events để kích hoạt pipeline tự động có điều kiện.

3.5 Inference

Table 6: Ánh xạ nhóm Inference.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Inference Service	Có	Model-server gateway + ML/DL serving workers; định tuyến theo phiên bản; xác thực JWT/API-Key; Prometheus metrics.	Nền tảng phục vụ suy luận hoạt động ổn định và tin cậy (đặc trưng V1).
Inference Engine Pool	Thiếu	Không sử dụng mô hình engine pool nạp động đa mô hình.	Không thuộc phạm vi kiến trúc hướng tới (V1/V3).
Runtime Model Monitor	Một phần	Prometheus theo dõi latency/request count; Redpanda event stream; Evidently tính toán data drift.	Còn thiếu: giám sát hiệu năng thực tế (F1/Recall trên nhãn trễ) và trôi dạt phân phối dự đoán.
Trigger	Một phần	Transactional Outbox tạo DriftRun tự động khi đủ ngưỡng số lượng bản ghi tích lũy.	Hiện chỉ kích hoạt tính toán drift, chưa kết nối tới luồng chẩn đoán hay quyết định CT.
Model Comparison Runner	Thiếu	Giao diện so sánh mô hình hiện là placeholder, chưa có engine so sánh đối kháng thực tế.	Khoảng trống trọng yếu —cần thiết để thẩm định candidate trước khi triển khai.
Decision Processor	Thiếu	Chưa có khối logic ra quyết định bảo trì mô hình.	Sẽ được hiện thực hóa qua module Chẩn đoán và RetrainingRequest.
XAI Module	Thiếu	Chưa tích hợp module giải thích mô hình (Explainable AI).	Thành phần mở rộng nghiên cứu, không thuộc phạm vi cốt lõi của đề tài.

3.6 Infrastructure & Supporting Services

Table 7: Ánh xạ nhóm Infrastructure & Supporting Services.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Resource Manager	Một phần	TrainingJob quản lý cấu hình tài nguyên (vcpu, memory_mb, GPU); Kubernetes resource limits.	Còn thiếu: cơ chế quan sát chi phí tính toán (cost observability) và metrics GPU chi tiết.
Communication Middleware	Có	Redpanda (tương thích Kafka) truyền nhận sự kiện suy luận; EventOutbox; Celery + Redis cho hàng đợi tác vụ.	Đạt chuẩn mực cao về tính ổn định và khả năng mở rộng.
Container Orchestrator	Có	Docker Compose cho môi trường phát triển cục bộ; Kubernetes/K3s cho môi trường sản xuất.	Đã đáp ứng trọn vẹn yêu cầu điều phối container.
Workflow Orchestrator	Có	Celery xử lý tác vụ bất đồng bộ; Argo Workflows/Kubeflow điều phối huấn luyện quy mô lớn; GitOps.	Hệ thống điều phối quy trình hoạt động hoàn chỉnh và linh hoạt.
Log Manager	Có	Bảng <code>paas_production_logs</code> ; nhật ký TrainingJobEvent; RegistryEvent; Transactional Outbox.	Hạ tầng quản lý log phân tầng và đảm bảo tính nhất quán dữ liệu.
API Gateway	Có	Model-server FastAPI gateway; Traefik reverse proxy; Django REST framework APIs tại Control Plane.	Đã hoàn thiện tốt khả năng định tuyến và kiểm soát truy cập an toàn.
MLOps User Interaction Manager	Một phần	Giao diện người dùng React/Vite; dashboard theo dõi drift; giao diện quản lý training/deployment.	Cần bổ sung dòng thời gian bằng chứng, hộp thư duyệt yêu cầu và giao diện so sánh mô hình.

3.7 Tổng hợp kết quả ánh xạ 39 thành phần

3.8 Nền tảng kiến trúc hiện hữu có giá trị kế thừa

- **Phân tách ranh giới các mặt phẳng (Planes Boundary):** Hệ thống phân định rõ ràng giữa Control Plane (điều phối vòng đời, xác thực đa tenant), Data Plane (phục vụ suy luận với độ trễ thấp) và Execution Plane (huấn luyện phi tập trung qua Docker hoặc Argo/Kubeflow). Trạng thái nghiệp vụ được bảo đảm tính nhất quán bền vững trên PostgreSQL và S3.
- **Tính bất biến của phiên bản mô hình (Version Immutability):** Mọi build thành công đều gắn liền với digest container image và artifact checksum cố định. Khái niệm Model Alias cho phép thăng cấp (promotion) hoặc thu hồi (rollback) linh hoạt mà không làm biến đổi lịch sử phiên bản.
- **Kênh truyền nhận sự kiện sản xuất an toàn (Production Event Pipeline):** Gateway truyền dữ liệu suy luận có phân quyền tenant/project sang Redpanda/Kafka; dịch vụ Consumer lưu vết vào bảng `paas_production_logs` và sử dụng Transactional Outbox pattern để tạo tín hiệu kích hoạt tính toán drift an toàn.
- **Quy trình giám sát trôi dạt dữ liệu (Drift Workflow):** Hỗ trợ liên kết model version với dữ liệu tham chiếu (reference dataset); Evidently tính toán chỉ số trôi dạt; callback cập nhật trạng thái DriftRun dựa trên UUID và idempotency key.
- **Hạ tầng huấn luyện linh hoạt:** Thực thể TrainingJob hỗ trợ cấu hình tài nguyên chi tiết (vCPU, RAM, GPU), cơ chế bảo mật capabilities-based và lưu vết bất biến mã nguồn (`code_snapshot_uri`) cùng dữ liệu

Table 8: Tổng hợp mức độ đáp ứng 39 thành phần chức năng theo từng nhóm.

Nhóm thành phần	Có	Một phần	Thiếu	Nhận định kiến trúc
Data Curation (5)	0	2	3	Nhóm yếu nhất; cần bổ sung Data Quality Checker và Feedback cho CT.
Storage & Versioning (10)	5	2	3	Model/Artifact Repository mạnh mẽ; thiếu Feedback Database liên kết.
ML Training (5)	1	2	2	ML Experiment Pipeline tốt; Continuous Training là trọng tâm phát triển.
CI/CD (5)	2	0	3	Đóng gói/triển khai tốt cho V1; thiếu pipeline builder/deployer cho V3.
Inference (7)	1	2	4	Serving gateway ổn định; thiếu Model Comparison Runner và performance monitoring.
Infrastructure & Supporting (7)	5	2	0	Hạ tầng nền tảng hoàn thiện; cần bổ sung giao diện tương tác nâng cao.
Tổng cộng (39)	14	10	15	Đánh giá bao quát đủ 39 thành phần nhằm minh bạch phạm vi và định vị rõ ranh giới nghiên cứu.

(data_snapshot_uri).

3.9 Phân tích các khoảng trống kiến trúc then chốt

Đối chiếu với yêu cầu của một hệ thống Continuous Training thích ứng, repository hiện tồn tại 6 khoảng trống cốt lõi:

1. **Hợp đồng dữ liệu sự kiện (Event Contract) chưa hoàn chỉnh:** Mặc dù schema consumer đã định nghĩa các trường confidence, latency_ms, status_code và request_id, nhưng gateway model-server hiện chỉ mới gửi dữ liệu features và output thô. Điểm số tự tin (confidence) và các thông số vận hành chưa được truyền tải vào event stream.
2. **Đứt gãy mối nối định danh dự đoán (Prediction ID Correlation):** Khóa định danh sự kiện id được sinh nội bộ tại gateway nhưng chưa được trả về cho phía client trong response. Do đó, client không có mã tham chiếu để gửi nhãn thực tế (ground truth) tương ứng sau này.
3. **Phạm vi giám sát dừng lại ở Data Drift:** Evidently hiện chỉ đo lường sự dịch chuyển phân phối dữ liệu đầu vào ($P(X)$), chưa hỗ trợ đánh giá chất lượng dữ liệu (data quality checks) cũng như chưa đo lường suy giảm hiệu năng thực tế ($P(Y|X)$, F1, Precision, Recall) do thiếu nhãn.
4. **Cơ chế kích hoạt thụ động (Passive Triggering):** Bảng paas_automatic_drift_outbox chỉ kích hoạt việc tạo một lượt chạy DriftRun khi tích lũy đủ số lượng bản ghi, hoàn toàn chưa có liên kết tự động tới luồng chẩn đoán hay yêu cầu tái huấn luyện.
5. **Quy trình đánh giá và thăng cấp thủ công:** Chức năng so sánh mô hình trên giao diện hiện chỉ hiển thị placeholder, thiếu một Evaluation Gate có thể tự động kiểm tra đối kháng (champion vs. candidate) trước khi quyết định triển khai.
6. **Chưa hỗ trợ phân tách lưu lượng (Traffic Splitting):** Gateway chỉ định tuyến toàn bộ 100% lưu lượng tới một phiên bản duy nhất, chưa hỗ trợ Shadow Mode (nhân bản request) hoặc Canary Rollout (chia tỉ lệ lưu lượng động).

4 Đối chiếu quy trình vòng đời (56 Process Steps)

Kết quả đối chiếu quy trình khẳng định một kết luận kiến trúc quan trọng: **hệ thống đã xây dựng hoàn chỉnh hạ tầng đường ống (Ingestion Plumbing), nhưng còn thiếu hoàn toàn khối trí tuệ ra quyết định (Adaptive Decision Loop)**. Việc chỉ gắn thêm một lệnh kích hoạt tái huấn luyện đơn giản sau khi phát hiện data drift sẽ không tạo nên một hệ thống CT tin cậy, bởi vì: (1) data drift không đồng nghĩa với suy giảm hiệu năng thực tế (có thể là benign drift), và (2) việc tự động huấn luyện lại trên dữ liệu bị lỗi hoặc nhãn nhiễu sẽ làm lặp lại hoặc

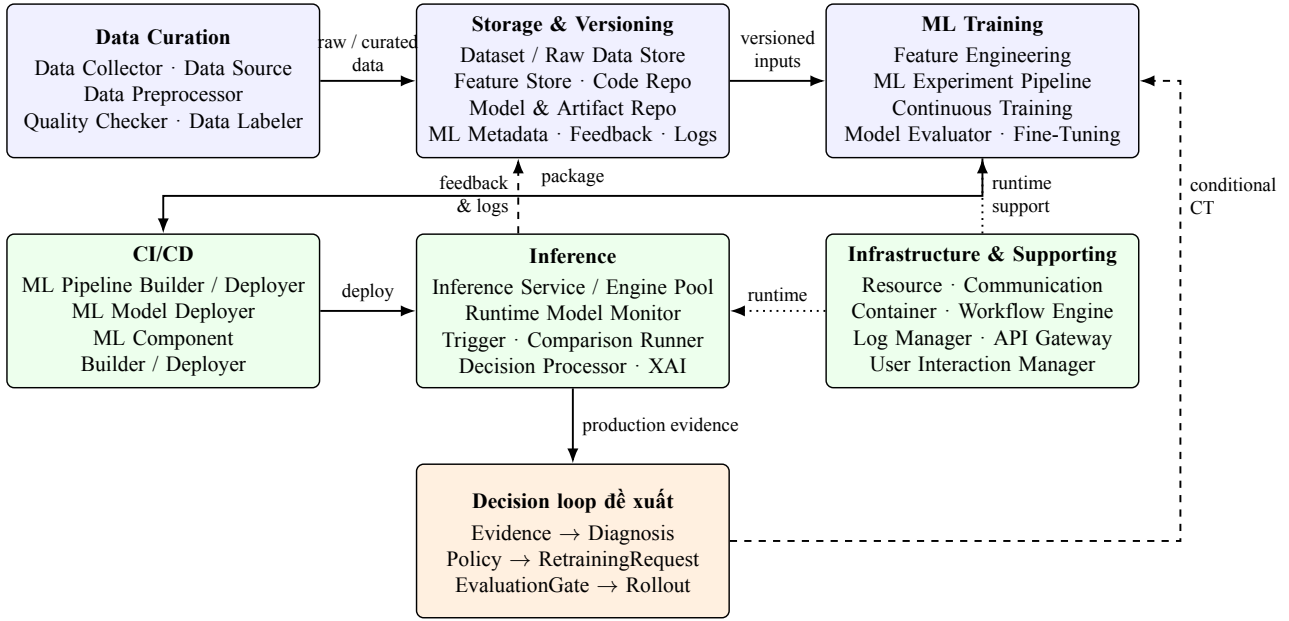


Figure 1: Bản đồ sáu nhóm component của paper và decision loop đề xuất cho repo. Màu xanh biểu diễn các nhóm nền tảng; màu cam là lớp cần xây dựng để chuyển từ V1 sang continuous training.

trầm trọng hóa sai số mô hình.

5 Hướng phát triển: Degradation-Aware Adaptive Model Maintenance

5.1 Luận đề nghiên cứu và Phạm vi

Luận đề nghiên cứu cốt lõi (Central Research Thesis):

“Data Drift $\neq$ Model Degradation $\neq$ Retraining Trigger. Một hệ thống Continuous Training tin cậy phải xác định chính xác nguyên nhân, mức độ nghiêm trọng (Severity) và độ tin cậy của bằng chứng trước khi quyết định hành động; không tự động tái huấn luyện theo quán tính chỉ dựa trên một cảnh báo drift đơn lẻ.”

Vòng lặp khép kín mục tiêu (Closed-loop Architecture):

Inference + Feedback → EvidenceWindow → DegradationEvent → Multi-signal Diagnosis → Risk/Policy Decision → Conditional CT / HITL / No-op → EvaluationGate → Shadow/Canary → Promotion / Rollback

Hệ thống phân tách rãnh mạch không gian hành động thành 5 nhánh can thiệp (Action Space):

1. **NO_OP / OBSERVE (Quan sát):** Ghi nhận sự kiện khi xảy ra trôi dạt lành tính (Benign Drift) hoặc mô hình hoạt động ổn định; tuyệt đối không tái huấn luyện nhằm tiết kiệm chi phí tài nguyên.
2. **DATA_ENGINEERING (Sự cố dữ liệu):** Đình chỉ toàn bộ tiến trình CT khi phát hiện lỗi schema, vi phạm miền giá trị hoặc mất tính tươi mới dữ liệu; chuyển giao cảnh báo cho đội ngũ kỹ thuật (**Engineering Incident**) để xử lý tận gốc rễ.
3. **INCREMENTAL_RETRAIN (Tái huấn luyện gia tăng):** Kích hoạt cập nhật mô hình trên cửa sổ dữ liệu mới khi trôi dạt có hại ở mức độ vừa phải (S_2) và đã có đủ mẫu nhãn kiểm chứng.
4. **FULL_RETRAIN (Tái huấn luyện toàn phần):** Kích hoạt quy trình huấn luyện lại toàn bộ khi suy giảm nghiêm trọng (S_3, S_4) và có sự dịch chuyển phân phối sâu rộng.
5. **HITL (Human-in-the-Loop):** Chuyển giao chuyên gia ML thẩm định và ra quyết định khi tín hiệu suy giảm mâu thuẫn, thiếu nhãn kiểm chứng hoặc suy giảm chưa rõ nguyên nhân.

Hình 2 biểu diễn thứ tự kiểm tra an toàn và các điểm rẽ nhánh trước khi cho phép một candidate đi vào CT.

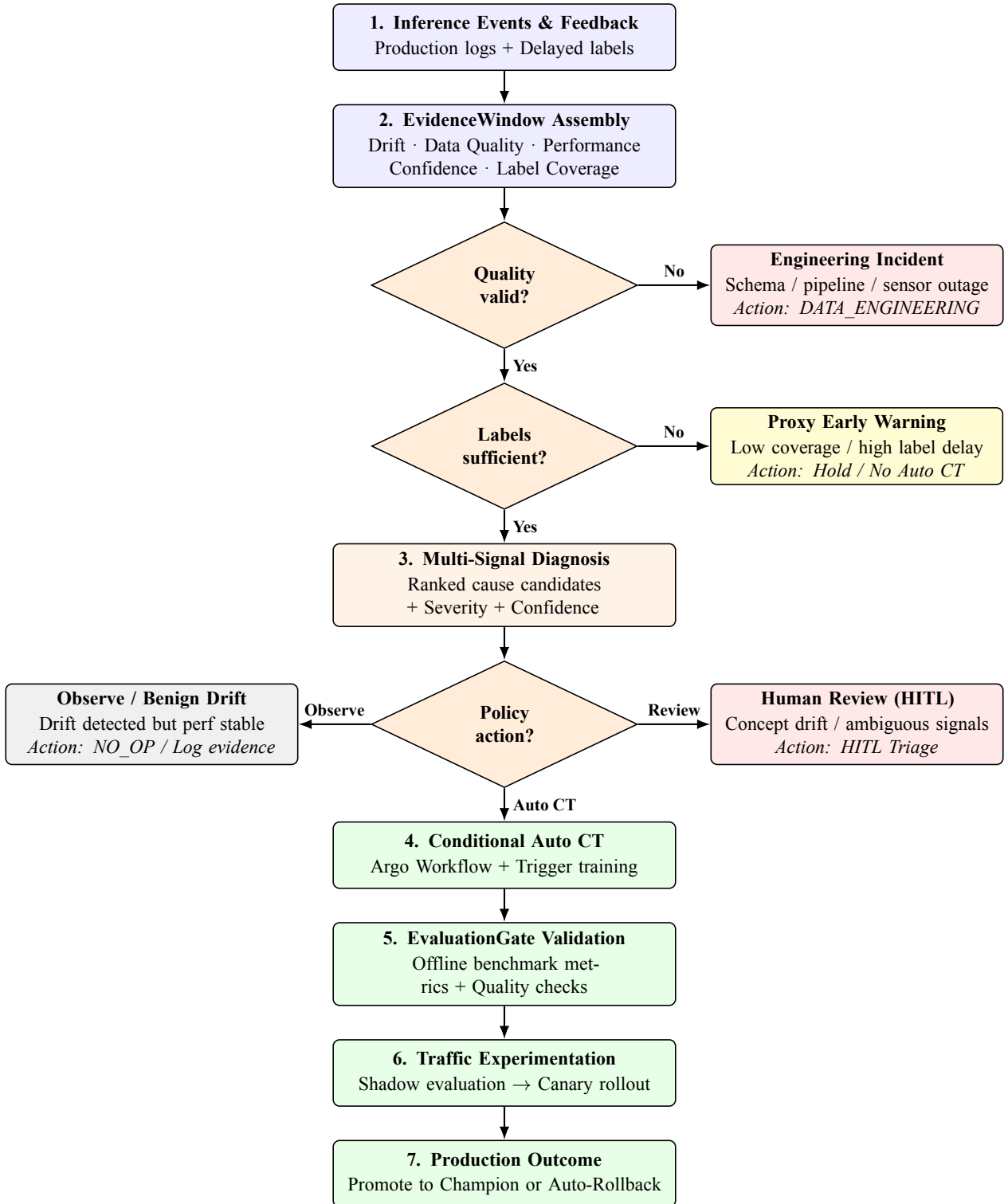


Figure 2: Workflow degradation-aware: Trục dọc thể hiện pipeline tự động hóa khép kín (happy path); các nhánh ngang thể hiện chốt an toàn và điểm can thiệp của con người/kỹ thuật khi thiếu bằng chứng.

Table 9: Hiện trạng triển khai các nhóm quy trình vòng đời then chốt [1].

Nhóm quy trình	Mức độ	Hiện trạng triển khai thực tế
Khởi tạo, đóng gói & đăng ký	Có	Upload code/data → build container image → đăng ký model version bất biến.
Thử nghiệm & huấn luyện	Một phần	Training job quản lý snapshot dữ liệu/mã nguồn; tracking metadata tích hợp qua MLflow.
Triển khai & phục vụ dự đoán	Có	Model build sẵn sàng được deploy; Gateway xác thực và proxy theo tenant/project/version.
Thu thập bằng chứng sản xuất	Một phần	Đã lưu vết features/prediction; cân bổ sung confidence, latency, prediction ID và feedback.
Phát hiện suy giảm (Detection)	Một phần	Đã có data drift và metrics hạ tầng; chưa chuẩn hóa prediction drift, data quality và performance.
Chẩn đoán nguyên nhân (Diagnosis)	Thiếu	Chưa có module suy luận nguyên nhân (cause candidates), đồ thị bằng chứng và giải thích.
Lựa chọn hành động & phê duyệt	Thiếu	Alert chưa chuyển đổi thành RetrainingRequest; thiếu cơ chế định tuyến Auto/Human/Engineering.
Tái huấn luyện, kiểm thử & thăng cấp	Một phần	Thực hiện thủ công; thiếu Evaluation Gate tự động, shadow/canary routing và auto-rollback.

5.2 Câu hỏi nghiên cứu và Giả thuyết khoa học

Để chứng minh đóng góp học thuật vượt ra ngoài phạm vi xây dựng tính năng kỹ thuật, quá trình thực nghiệm sẽ tập trung kiểm chứng câu hỏi nghiên cứu trung tâm:

“Khi mô hình sản xuất xuất hiện nhiều tín hiệu suy giảm, làm thế nào hệ thống xác định chính xác nguyên nhân, mức độ nghiêm trọng (Severity) và độ tin cậy của bằng chứng để lựa chọn hành động bảo trì phù hợp, an toàn và tiết kiệm chi phí?”

Bốn giả thuyết khoa học (Research Hypotheses) được xây dựng phục vụ kiểm định thống kê:

- **H1 (Hiệu quả chi phí và phục hồi):** Chính sách thích ứng dựa trên nhận thức suy giảm giảm thiểu có ý nghĩa thống kê số lần tái huấn luyện không cần thiết (unnecessary retraining) so với các baseline định kỳ (periodic) và dựa trên cảnh báo drift đơn lẻ (alert-driven), trong khi vẫn duy trì năng lực phục hồi hiệu năng tương đương khi gặp trôi dạt có hại.
- **H2 (Độ chính xác quyết định):** Khối chẩn đoán nguyên nhân đa tín hiệu (Multi-Signal Diagnosis) làm giảm tỷ lệ ra quyết định tái huấn luyện sai lầm (false retraining decisions) so với cơ chế kích hoạt chỉ dựa trên một detector duy nhất.
- **H3 (An toàn tự động hóa với HITL):** Cơ chế can thiệp nhận thức rủi ro kết hợp vòng lặp chuyên gia (HITL) triệt tiêu các hành vi tự động hóa mất an toàn (unsafe automation) trong các kịch bản suy giảm mơ hồ hoặc khi bằng chứng nhận chưa đạt độ tin cậy thống kê.
- **H4 (Tối ưu hóa tổng thể):** Phương pháp đề xuất cải thiện vượt trội tỷ lệ đánh đổi (trade-off) giữa mức độ phục hồi hiệu năng, chi phí tính toán hạ tầng và khối lượng can thiệp thủ công so với các chính sách CT truyền thống.

5.3 Khung phân loại nguyên nhân và Mức độ nghiêm trọng (Taxonomy & Severity)

Khối chẩn đoán không chỉ trả về cờ boolean $\text{Drift} \in \{\text{True}, \text{False}\}$, mà xuất ra cấu trúc chẩn đoán chính xác gồm 3 thành tố: Nguyên nhân (Cause), Mức độ nghiêm trọng (Severity), và Độ tin cậy (Confidence).

5.3.1 Hệ thống phân loại nguyên nhân (Cause Taxonomy)

- **NORMAL / STABLE:** Hệ thống và dữ liệu ổn định, không có biến động bất thường.
- **BENIGN_DRIFT:** Dữ liệu đầu vào $P(X)$ thay đổi nhưng phân phối điều kiện $P(Y|X)$ và hiệu năng thực tế không suy giảm.

- **HARMFUL_DRIFT**: Dữ liệu đầu vào biến đổi kèm theo suy giảm hiệu năng thực tế có ý nghĩa thống kê trên tập nhân xác thực.
- **DATA_QUALITY_FAILURE**: Lỗi schema, sai lệch miền giá trị, missing rate tăng vọt hoặc mất tính tươi mới dữ liệu.
- **CONCEPT_SHIFT / UNEXPLAINED**: Hiệu năng thực tế suy giảm nhưng phân phối dữ liệu đầu vào $P(X)$ không đổi, đòi hỏi chuyên gia phân tích.
- **INSUFFICIENT_EVIDENCE**: Tỷ lệ phủ nhãn $r_{\text{labels}} < r_{\text{min}}$ hoặc số lượng mẫu chưa đủ để khẳng định suy giảm.
- **MIXED / AMBIGUOUS**: Tín hiệu mâu thuẫn giữa nhiều detector hoặc đồng thời xảy ra sự cố hạ tầng và trôi dạt dữ liệu.

5.3.2 Thang đo mức độ nghiêm trọng (Severity Levels)

Mức độ nghiêm trọng được lượng hóa thành 5 cấp bậc S_0 – S_4 dựa trên độ lớn trôi dạt (d_{drift}), mức suy giảm hiệu năng (Δ_{perf}), khoảng tin cậy thống kê (CI_{perf}) và độ phủ nhãn:

- S_0 (**Normal**): Hoạt động bình thường; $\Delta_{\text{perf}} \approx 0$.
- S_1 (**Benign / Low**): Trôi dạt đầu vào nhẹ nhưng hiệu năng ổn định, hoặc cảnh báo sớm từ phân phối confidence khi chưa có nhãn.
- S_2 (**Moderate**): Suy giảm hiệu năng từ 3%–8% kèm $d_{\text{drift}} > \theta_{\text{drift}}$ và nhãn kiểm chứng đầy đủ.
- S_3 (**Severe**): Suy giảm hiệu năng $> 8\%$ trên diện rộng hoặc vi phạm chất lượng dữ liệu nghiêm trọng.
- S_4 (**Critical**): Mô hình sụp đổ hoàn toàn hoặc vi phạm guardrails an toàn trên môi trường sản xuất.

Table 10: Bảng phân loại chính sách ra quyết định thích ứng Bằng chứng – Hành động.

Mẫu hình bằng chứng	Chẩn đoán & Severity	Hành động đề xuất	Điều kiện kích hoạt
d_{drift} cao; Quality đạt; Nhãn đủ; F1 giảm vừa	HARMFUL_DRIFT (S_2 Moderate)	INCREMENTAL_CT	Cooldown OK $\wedge$ Budget OK $\wedge$ $r_{\text{labels}} \geq r_{\text{min}}$
d_{drift} cao; F1 giảm mạnh $> 8\%$; Nhãn đủ	HARMFUL_DRIFT (S_3, S_4 Severe)	FULL_CT	Cooldown OK $\wedge$ Budget OK $\wedge$ Vượt EvaluationGate
Vi phạm Quality (schema, range, missing cao)	DATA_QUALITY_FAIL (S_3 Severe)	DATA_ENGINEERING	Định chỉ CT; phát cảnh báo Engineering Incident
F1 giảm; d_{drift} thấp hoặc mâu thuẫn	CONCEPT_SHIFT (S_3 Severe)	HITL	Chuyển giao chuyên gia phân tích lát cắt (slice)
d_{drift} cao nhưng F1 ổn định (đủ nhãn)	BENIGN_DRIFT (S_1 Low)	NO_OP / OBSERVE	Ghi nhận log bằng chứng; không huấn luyện lại
Tín hiệu lệch nhưng $r_{\text{labels}} < r_{\text{min}}$	INSUFFICIENT_EVID (S_1 Low)	NO_OP / PROXY_HOLD	Tăng tỷ lệ lấy mẫu; cấm tự động kích hoạt CT

5.4 Thuật toán chẩn đoán và Cấu trúc dữ liệu đầu ra

5.4.1 Cấu trúc dữ liệu chẩn đoán chuẩn hóa (Structured JSON Output)

Kết quả của khối chẩn đoán được chuẩn hóa dưới dạng JSON schema có thể giải thích được và lưu vết kiểm toán:

```
{
  "event_id": "deg_evt_8f1a2c",
  "cause": "HARMFUL_DRIFT",
  "severity": "S2_MODERATE",
  "confidence": 0.94,
  "evidence": {
    "psi": 0.32,
    "f1_delta": -0.075,
    "label_coverage": 0.86,
    "quality_passed": true
  },
  "recommended_action": "INCREMENTAL_RETRAIN",
  "action_constraints": { "cooldown_passed": true, "budget_ok": true }
}
```

Thuật toán 1: Chẩn đoán nguyên nhân suy giảm và xác định hành động bảo trì

Đầu vào (Input): Thực thể EvidenceWindow $E = \{d_{\text{drift}}, q_{\text{quality}}, \Delta_{\text{perf}}, \Delta_{\text{conf}}, r_{\text{labels}}, n, CI_{\text{perf}}\}$

Bước 1 (Kiểm định chất lượng dữ liệu - Data Quality Gate):

If $q_{\text{quality}} < \theta_{\text{quality}}$ then

Return Output(DATA_QUALITY_FAILURE, Severity: S_3 , Action: DATA_ENGINEERING, Conf: High)

Bước 2 (Kiểm tra độ phủ nhãn - Label Sufficiency Gate):

If $r_{\text{labels}} < r_{\text{min}}$ or $n < n_{\text{min}}$ then

If Δ_{conf} lệch chuẩn có ý nghĩa then

Return Output(INSUFFICIENT_EVIDENCE, Severity: S_1 , Action: NO_OP_HOLD, Conf: Low)

Else

Return Output(NORMAL, Severity: S_0 , Action: NO_OP, Conf: High)

Bước 3 (Kiểm định trôi dạt có hại - Harmful Drift Gate):

If $d_{\text{drift}} > \theta_{\text{drift}}$ and $\Delta_{\text{perf}} < \theta_{\text{perf}}$ and Cận dưới CI_{perf} khẳng định suy giảm then

If $|\Delta_{\text{perf}}| > 0.08$ then Sev $\leftarrow S_3$; Act $\leftarrow$ FULL_RETRAIN

Else Sev $\leftarrow S_2$; Act $\leftarrow$ INCREMENTAL_RETRAIN

If (Cooldown OK $\wedge$ Budget OK) then

Return Output(HARMFUL_DRIFT, Severity: Sev, Action: Act, Conf: High)

Else

Return Output(HARMFUL_DRIFT, Severity: Sev, Action: HITL, Conf: High)

Bước 4 (Trôi dạt lành tính - Benign Drift Gate):

If $d_{\text{drift}} > \theta_{\text{drift}}$ and $|\Delta_{\text{perf}}| \approx 0$ then

Return Output(BENIGN_DRIFT, Severity: S_1 , Action: NO_OP, Conf: High)

Bước 5 (Suy giảm hiệu năng chưa rõ nguyên nhân - Concept Shift):

If $\Delta_{\text{perf}} < \theta_{\text{perf}}$ and $d_{\text{drift}} \leq \theta_{\text{drift}}$ then

Return Output(CONCEPT_SHIFT, Severity: S_3 , Action: HITL, Conf: Medium)

Mặc định (Fallback):

Return Output(MIXED_AMBIGUOUS, Severity: S_2 , Action: HITL, Conf: Low)

5.5 Chiến lược xử lý nhãn trễ (Delayed-Label Handling Strategy)

Nhãn thực tế (ground-truth labels) thường có độ trễ lớn so với thời điểm suy luận, tạo nên điểm nghẽn nghiêm trọng nhất trong việc giám sát hiệu năng mô hình. Hệ thống đề xuất 5 nguyên tắc xử lý nhãn trễ:

- Chuỗi liên kết mã định danh dự đoán (End-to-end Prediction ID):** Gateway model-server tạo mã định danh duy nhất (prediction_id) và trả về cho client trong HTTP response, đồng thời gắn kèm mã này vào event stream lưu trữ. Client sử dụng chính prediction_id để gửi nhãn thực tế qua Feedback API. Đây là **điều kiện tiên quyết** để thực hiện phép join giữa dự đoán và nhãn thực tế.
- Cửa sổ trượt gắn mốc thời gian nhãn (Watermark-based Sliding Window):** Phép tính toán hiệu năng (F1, Precision, Recall) chỉ được thực hiện trên cửa sổ thời gian $[t - W, t]$ khi tỷ lệ phủ nhãn đạt ngưỡng tin

cây tối thiểu:

$$r_{\text{labels}} = \frac{|\text{Dự đoán đã có nhãn}|}{|\text{Tổng số lượt dự đoán}|} \geq r_{\text{min}}$$

Nếu $r_{\text{labels}} < r_{\text{min}}$, hệ thống ghi nhận trạng thái chưa đủ nhãn (INSUFFICIENT_LABELS) và tuyệt đối không nới suy hiệu năng giả định để auto-retrain.

3. **Chỉ số đại diện khi thiếu nhãn (Proxy Signals):** Khi nhãn chưa sẵn sàng, hệ thống dựa vào các tín hiệu đại diện: độ trôi dạt phân phối đầu ra $P(\hat{Y})$, độ dịch chuyển điểm tự tin (calibrated confidence drift) và biến động độ trễ để phát cảnh báo sớm, không dùng để kích hoạt CT trực tiếp.
4. **Phân tầng độ tin cậy của nhãn (Label Trust Levels):** Mỗi bản ghi nhãn phản hồi đều lưu kèm nguồn thu thập (label_source: người dùng cuối, chuyên gia gán nhãn, hay pipeline tự động) và mức độ tin cậy (trust_level), ngăn ngừa việc trộn lẫn nhãn nhiều vào dữ liệu huấn luyện mới.
5. **Ước lượng thống kê với độ phủ nhãn một phần:** Khi chỉ có một phần dự đoán được gán nhãn (ví dụ 30%), hiệu năng được tính toán kèm khoảng tin cậy 95% (CI_{perf}). Công quyết định chỉ kích hoạt CT khi cận dưới của khoảng tin cậy xác nhận sự suy giảm thực sự.

5.6 Mô hình dữ liệu trạng thái (Minimal State Model)

Để hiện thực hóa vòng lặp khép kín, Control Plane cần bổ sung 8 thực thể dữ liệu bất biến:

Table 11: Các thực thể dữ liệu cốt lõi bổ sung vào Control Plane.

Thực thể (Entity)	Thuộc tính then chốt	Mục đích kiến trúc & Nghiệp vụ
PredictionRecord	prediction_id, request_id, tenant, project, version, timestamp, features, prediction, confidence, latency, status	Ghi nhận từng giao dịch suy luận và thiết lập khóa liên kết (correlation key) cho dữ liệu phản hồi.
Feedback	feedback_id, prediction_id, ground_truth, label_timestamp, label_source, trust_level, correction_version	Lưu vết nhãn thực tế trả về từ client; hỗ trợ nhiều lượt phản hồi và đính chính nhãn (correction).
EvidenceWindow	model_version, window_start, window_end, baseline_ref, metric_values (drift, quality, perf, conf), slices, label_coverage, policy_version	Lưu trữ bằng chứng vận hành bất biến theo cửa sổ thời gian phục vụ chẩn đoán và kiểm toán.
DegradationEvent	evidence_window_ids, severity, affected_component, deduplication_key, status, created_at	Chuẩn hóa sự kiện cảnh báo suy giảm với tính chất bất biến và chống trùng lặp (idempotency).
Diagnosis	degradation_event_id, ranked_causes (JSON), severity_level, top_confidence, explanation, lineage_snapshot, policy_version	Tách biệt tường minh khâu suy luận nguyên nhân gốc rễ khỏi khâu phát hiện cảnh báo thô.
RetrainingRequest	diagnosis_id, strategy (incremental/full/hitl/eng), data_snapshot_uri, config_version, budget_limit, cooldown_until, status, approved_by	Đơn vị điều phối vòng đời tái huấn luyện thay thế cho việc gọi trực tiếp training job từ drift run.

Thực thể (Entity)	Thuộc tính then chốt	Mục đích kiến trúc & Nghiệp vụ
EvaluationGate	candidate_version, champion_version, offline_metrics, calibration_score, latency_p95, error_rate, data_quality_passed, decision (pass/fail/hold)	Cổng thẩm định đối kháng nghiêm ngặt trước khi cho phép mô hình ứng viên tiếp cận môi trường sản xuất.
TrafficExperiment	experiment_type (shadow/canary), candidate_version, champion_version, hash_seed, weight_schedule, guardrails, current_weight, rollback_reason	Điều phối và ghi nhận toàn bộ quá trình thử nghiệm lưu lượng thực tế và cơ chế thu hồi tự động.

Một prediction có thể chưa có label hoặc có nhiều lần label/correction; vì vậy PredictionRecord và Feedback nên là hai bảng có quan hệ một-nhiều, kèm unique/idempotency constraint cho nguồn feedback và version correction.

6 Ranh giới tự động hoá và Quy trình kiểm thử thăng cấp

Table 12: Ma trận phân định ranh giới tự động hóa (Auto vs. Human vs. Engineering).

Cấp độ tự động	Quyền hạn của hệ thống	Ranh giới cấm tự động (Cần con người/CI)
Observe / No Action	Ghi nhận sự kiện, tăng tỷ lệ lấy mẫu, theo dõi cửa sổ tiếp theo.	Tuyệt đối không kích hoạt huấn luyện (tránh training storm).
Conditional Auto CT	Snapshot dữ liệu được phê duyệt, kích hoạt huấn luyện, kiểm thử qua EvaluationGate và đăng ký candidate.	Không tự ý thay đổi định nghĩa nhãn, cấu trúc feature, mã nguồn hay kiến trúc mô hình.
Human-approved CT	Chuẩn bị đầy đủ báo cáo chẩn đoán, vết nguồn gốc (lineage) và kế hoạch huấn luyện ứng viên.	Không tự quyết định đánh đổi rủi ro kinh doanh hoặc tiếp tục khi nhãn mâu thuẫn.
Engineering Incident	Tạo issue cảnh báo kèm bằng chứng audit trail, chuyển giao đội ngũ kỹ thuật.	Không tự can thiệp chỉnh sửa pipeline, dependency thư viện, mã ETL hay hạ tầng runtime.

6.1 Kiến trúc thử nghiệm lưu lượng thực tế (Shadow & Canary Architecture)

Quy trình thăng cấp (promotion) trong kiến trúc đề xuất tuân thủ phương pháp kiểm định đa tầng lũy tiến thay vì hoán đổi trực tiếp sau huấn luyện:

- Chế độ nhân bản lưu lượng (Shadow Mode - Phase 5a):** Mô hình ứng viên (Candidate) nhận một bản sao lưu lượng thực tế từ gateway mà không làm ảnh hưởng tới phản hồi trả về người dùng (Champion vẫn trả response). Dự đoán của candidate được ghi nhận độc lập vào `paas_shadow_logs` để phục vụ so sánh đối kháng trên cùng request ID.
- Chế độ lưu lượng tăng dần (Canary Rollout - Phase 5b):** Định tuyến lưu lượng thực tế dựa trên băm nhất quán (consistent hashing) theo khóa phân nhóm ổn định (User/Session ID). Tỷ lệ lưu lượng được nâng dần theo các mốc 5% → 10% → 25% → 50% → 100% khi và chỉ khi thỏa mãn đầy đủ các chỉ số rào chắn an toàn (Guardrails).
- Tiêu chí rào chắn an toàn (Guardrail Evaluation):** Đánh giá liên tục trên cửa sổ trượt có kiểm định thống kê:

- Tỷ lệ lỗi (Error rate) của Candidate $\leq$ Champion $+\epsilon$
- Độ trễ p95 của Candidate $\leq$ Champion $\times 1.2$
- Khoảng cách phân phối dự đoán (KL-Divergence) $\leq \theta_{\text{dist}}$
- Hiệu năng thực tế (khi có nhãn) của Candidate $\geq$ Champion $-\delta$

4. **Thu hồi tự động (Automated Rollback):** Tự động ngắt toàn bộ lưu lượng của Candidate và chuyển đổi Blue-Green về Champion cũ khi vi phạm guardrail trong k cửa sổ liên tiếp, đồng thời lưu vết rollback_reason vào thực thể TrafficExperiment.

Hình 3 mô tả trực quan quy trình kiểm định thăng cấp lũy tiến này.

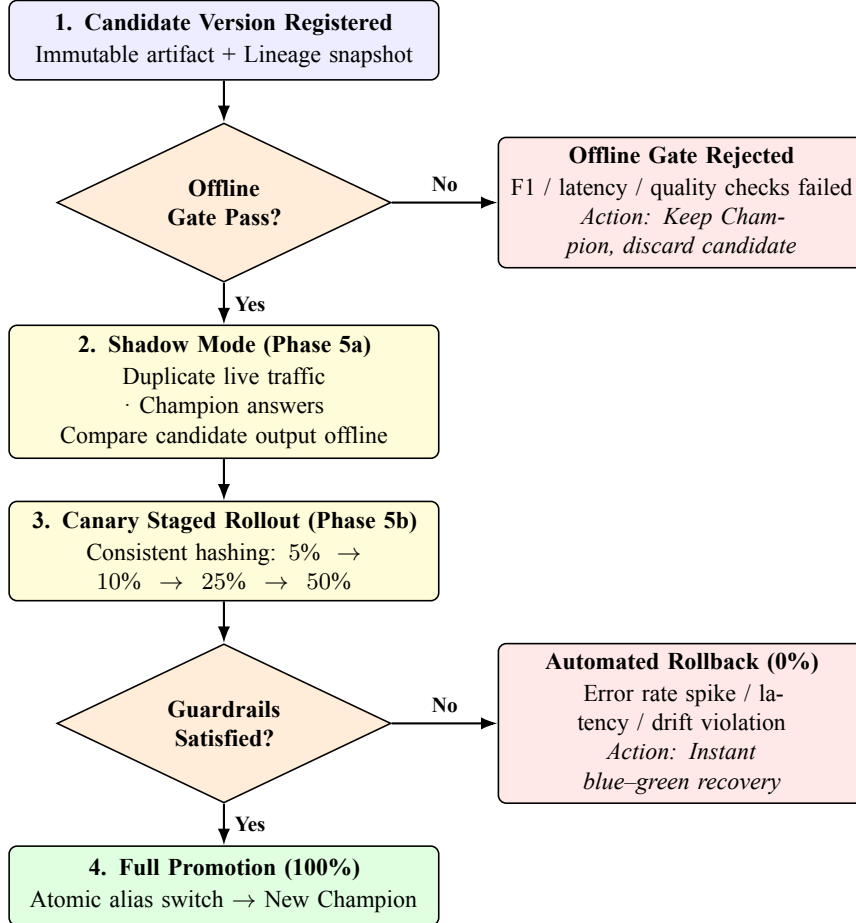


Figure 3: Protocol champion-challenger đề xuất: Trực dọc thể hiện các tầng kiểm thử lũy tiến (Offline Gate → Shadow → Canary → Full Promotion); nhánh ngang thể hiện điểm dừng an toàn và cơ chế rollback tự động.

7 Lộ trình hiện thực hoá và Kế hoạch nghiên cứu

Lộ trình thực hiện được thiết kế kéo dài 26 tuần, phân bổ thành 6 giai đoạn cấu trúc chặt chẽ nhằm bảo đảm tính sẵn sàng nghiên cứu (Research-Ready) và tối ưu hóa nguồn lực:

Table 13: Lộ trình nghiên cứu và phát triển 26 tuần.

Giai đoạn	Mục tiêu trọng tâm	Sản phẩm bàn giao (Deliverables)	Tiêu chí hoàn thành (Exit Criteria)
-----------	--------------------	----------------------------------	-------------------------------------

— Giai đoạn 1: Nền tảng Thực nghiệm & Vertical Slice PoC —

Giai đoạn	Mục tiêu trọng tâm	Sản phẩm bàn giao (Deliverables)	Tiêu chí hoàn thành (Exit Criteria)
Stage 1 (Tuần 1–4)	Đóng băng giao thức nghiên cứu & Lát cắt dọc	1a (Tuần 1–2): Bộ dataset tabular chuẩn, framework tạo kịch bản trôi dạt (Scenario Injector) gắn sẵn Ground-truth Cause, danh sách 6 baseline. 1b (Tuần 3–4): Lát cắt dọc tích hợp khép kín trên kịch bản S_2 : Inject drift → Detect → Argo Trigger → Train → Gate → Promote.	Giao thức benchmark được kiểm soát phiên bản (Git/S3 manifest); luồng dữ liệu khép kín thông suốt giữa các service.
— Giai đoạn 2: Hạ tầng Dữ liệu Bằng chứng & Nhãn trễ —			
Stage 2 (Tuần 5–9)	Hạ tầng Evidence & Feedback Store	2a (Tuần 5–6): Hoàn chỉnh event contract gửi prediction_id, confidence, latency, status. 2b (Tuần 7–8): Xây dựng Feedback API + Store, thực thể PredictionRecord, kiểm thử phép join nhãn theo watermark. 2c (Tuần 9): Snapshot dữ liệu bất biến, thực thể EvidenceWindow, kiểm định Data Quality.	Thực hiện thành công phép join prediction–feedback trên dữ liệu mẫu; truy vết trọn vẹn từ model → data → code.
— Giai đoạn 3: Chẩn đoán Suy giảm Đa tín hiệu —			
Stage 3 (Tuần 10–13)	Phân loại nguyên nhân & Định lượng Severity	3a (Tuần 10–11): Mở rộng Evidently đo prediction drift, data quality checks, slice monitoring. 3b (Tuần 12): Giám sát hiệu năng F1/Recall trên cửa sổ trượt watermark khi $r_{\text{labels}} \geq r_{\text{min}}$. 3c (Tuần 13): Hiện thực hóa bộ luật phân loại 7 nhóm nguyên nhân, định lượng Severity S_0 – S_4 , thực thể DegradationEvent.	Phân loại chính xác các dạng bất thường (S_0 – S_3 , S_{10} , S_{11}); đo lường được Precision/Recall của khâu phát hiện.
— Giai đoạn 4: Trí tuệ Ra quyết định Thích ứng (Research Core) —			

Giai đoạn	Mục tiêu trọng tâm	Sản phẩm bàn giao (Deliverables)	Tiêu chí hoàn thành (Exit Criteria)
Stage 4 (Tuần 14–17)	Chính sách can thiệp thích ứng & Vòng lặp HITL	4a (Tuần 14–15): Module Policy ánh xạ Evidence → Action Space (No-op, Data Eng, Incremental CT, Full CT, HITL). 4b (Tuần 16): Thực thể RetrainingRequest, chính sách cooldown, ngân sách tính toán (Budget Cap). 4c (Tuần 17): Giao diện duyệt yêu cầu (Approval Inbox), lưu vết kiểm toán quyết định (Decision Audit Trail).	Kiểm thử tự động chứng minh hệ thống không tự ý gọi huấn luyện khi xảy ra Benign Drift hoặc Data Quality Failure.
— Giai đoạn 5: Vòng đời Tái huấn luyện An toàn & Cổng Thẩm định —			
Stage 5 (Tuần 18–23)	Điều phối Argo Workflow & Cổng đối kháng	5a (Tuần 18–20): Kích hoạt Argo Workflow huấn luyện candidate có giới hạn tài nguyên, thực thể EvaluationGate, đăng ký Model Registry. 5b (Tuần 21–22): Điều phối Shadow Mode tại gateway, lưu log <code>paas_shadow_logs</code> , đo đặc sai lệch. 5c (Tuần 23, Stretch): Canary weighted routing, rào chắn guardrails và cơ chế tự động rollback.	Candidate có lineage bất biến và vượt qua cổng kiểm định đối kháng đa tầng trước khi được thăng cấp.
— Giai đoạn 6: Đánh giá Thực nghiệm Tổng thể & Công bố Khoa học —			
Stage 6 (Tuần 24–26)	Đánh giá Benchmark Toàn diện & Bài báo	Chạy toàn bộ 12 kịch bản (S_0 – S_{11}) với 6 baseline (B_0 – B_5), phân tích Ablation (A_0 – A_6), độ nhạy (sensitivity), chi phí tài nguyên, nỗ lực HITL và hoàn thiện bài báo khoa học.	Kết quả có thể tái lập hoàn toàn (reproducible); bộ dữ liệu, mã nguồn và báo cáo kỹ thuật được đóng gói chuẩn mực.

7.1 Thứ tự ưu tiên triển khai (Implementation Priorities)

Để tránh phân tán nguồn lực vào các tính năng kỹ thuật phụ trợ, quy trình cài đặt tuân thủ nghiêm ngặt 13 bước tuần tự:

- Prediction ID + Feedback API →
- Delayed Label Watermark + EvidenceWindow →
- Data Quality + Drift + Performance →
- DegradationEvent →
- Diagnosis Algorithm (Cause + Severity) →
- RetrainingRequest + Policy Engine →
- Argo Trigger →
- Training Execution →
- EvaluationGate →
- Model Registry →
- Shadow Mode →
- Benchmark Suite (S_0 – S_{11}) →
- Ablation & Sensitivity Analysis →
- Paper Packaging.

Các thành phần giao diện người dùng nâng cao hoặc điều phối Canary phức tạp chỉ được xem xét sau khi đã hoàn thành toàn bộ phép đo thực nghiệm cốt lõi.

7.2 Phân bổ trách nhiệm theo phân hệ dịch vụ (Subsystem Ownership)

- **model-server + consumer:** Hoàn chỉnh event contract (gửi `prediction_id`, `confidence`, `latency`, `status`); consumer lưu vết bền vững vào PostgreSQL và quản lý transactional outbox.
- **control plane:** Bổ sung 8 thực thể dữ liệu mới, cung cấp Feedback API, quản lý logic chẩn đoán nguyên nhân/severity, chính sách phê duyệt và kích hoạt Argo Event.
- **evidently service:** Mở rộng từ data drift sang prediction drift, kiểm định chất lượng dữ liệu và tính toán chỉ số hiệu năng trên nhãn thực tế theo watermark.
- **argo workflows + kubeflow runner:** Nhận snapshot dữ liệu bất biến, thực thi huấn luyện có giới hạn tài nguyên và trích xuất chỉ số kiểm định chuẩn hóa vào MLflow.
- **registry + deployment + GitOps:** Quản lý metadata candidate/champion, điều phối cấu hình định tuyến lưu lượng shadow và giám sát rào chắn an toàn.
- **web frontend (tối giản):** Xây dựng dòng thời gian bằng chứng (Evidence Timeline) và hộp thư phê duyệt (Approval Inbox) phục vụ vòng lặp HITL.

8 Thiết kế thực nghiệm và Đánh giá Benchmark

Để khẳng định tính đóng góp khoa học, bộ benchmark thực nghiệm được thiết kế theo nguyên tắc **nguyên nhân suy giảm đã biết trước (Ground-truth Cause)**, vượt ra ngoài việc chỉ quan sát nhãn bài toán máy học đơn thuần.

8.1 Không gian kịch bản thực nghiệm (Benchmark Scenarios)

Table 14: Danh mục các kịch bản thực nghiệm chuẩn hóa.

Mã	Phân tầng	Bản chất kịch bản (Scenario Injection)	Hành động kỳ vọng chuẩn xác
S0	Cốt lõi	Môi trường ổn định, không có suy giảm (Stable)	Không can thiệp (NO_OP)
S1	Cốt lõi	Trôi dạt lành tính: $P(X)$ đổi nhưng $P(Y X)$ ổn định	Cảnh báo, theo dõi (NO_OP / OBSERVE)
S2	Cốt lõi	Trôi dạt có hại: $P(X)$ đổi kéo theo F1 giảm, đủ nhãn	Tái huấn luyện gia tăng (INCREMENTAL_RETRAIN)
S3	Cốt lõi	Sự cố chất lượng dữ liệu: lỗi schema, range, missing	Đình chỉ CT, mở DATA_ENGINEERING
S4	Quan trọng	Dịch chuyển khái niệm/nhãn: $P(Y X)$ đổi đột ngột	Phê duyệt chuyên gia (HITL)
S9	Quan trọng	Tín hiệu hỗn hợp: vừa trôi dạt dữ liệu vừa trễ nhãn	Phân tích chuyên sâu (HITL)
S10	Quan trọng	Cảnh báo trôi dạt giả (False Drift Signal)	Nhận diện lành tính, không CT thừa
S11	Quan trọng	Nhãn trễ / Chưa đủ độ phủ ($r_{\text{labels}} < r_{\text{min}}$)	Tạm giữ (PROXY_HOLD), cấm Auto CT
S5	Mở rộng	Dữ liệu huấn luyện bị lão hóa dần theo thời gian	Tái huấn luyện toàn phần (FULL_RETRAIN)
S6	Mở rộng	Suy giảm cấu hình siêu tham số (HPO degradation)	Cập nhật không gian tìm kiếm HPO
S7	Mở rộng	Lỗi mã nguồn xử lý đặc trưng (Feature code bug)	Sửa đổi qua luồng CI/CD kỹ thuật
S8	Mở rộng	Sự cố hạ tầng phục vụ (OOM, nghẽn mạng)	Thu hồi tự động hoặc mở rộng tài nguyên

Mỗi kịch bản được tạo thông qua bộ sinh dữ liệu (Scenario Injector) với nhãn nguyên nhân suy giảm thực tế (`ground_truth_cause`), cho phép đánh giá định lượng ma trận nhầm lẫn (Confusion Matrix) của thuật toán chẩn

đoán.

8.2 Các phương pháp đối chứng (Baselines)

Quá trình thực nghiệm so sánh phương pháp đề xuất với 6 đường cơ sở chuẩn mực:

- **B0 – Static / No Retraining:** Mô hình giữ nguyên không tái huấn luyện trong suốt vòng đời; đo lường mức độ suy thoái tự nhiên.
- **B1 – Periodic Retraining:** Tái huấn luyện cố định theo lịch trình định kỳ (ví dụ: hàng tuần hoặc hàng tháng) bất kể có trôi dạt hay không.
- **B2 – Single-Signal Drift-Triggered CT:** Tự động tái huấn luyện ngay khi có bất kỳ cảnh báo data drift đơn lẻ nào vượt ngưỡng (đại diện cho cách tiếp cận phổ biến trong công nghiệp).
- **B3 – Performance-Triggered CT:** Chỉ kích hoạt tái huấn luyện khi hiệu năng kiểm chứng thực tế bị suy giảm.
- **B4 – Always Retrain:** Huấn luyện lại liên tục sau mỗi lô dữ liệu mới (đóng vai trò đường cơ sở trần về chi phí tài nguyên).
- **B5 – Proposed Degradation-Aware Adaptive CT:** Chuỗi xử lý hoàn chỉnh Detection → Diagnosis → Severity → Risk Policy → Conditional CT / HITL → Evaluation Gate.

8.3 Hệ thống chỉ số đánh giá đa chiều (Evaluation Metrics)

Table 15: Khung chỉ số đánh giá thực nghiệm toàn diện.

Chiều cạnh đánh giá	Tập chỉ số đo lường chi tiết
Chất lượng phát hiện	Precision, Recall, F1 của cảnh báo; False Positive Rate; độ trễ phát hiện (Detection Delay).
Chất lượng chẩn đoán	Độ chính xác chẩn đoán nguyên nhân (Diagnosis Accuracy); Macro-F1; Confusion Matrix giữa 7 loại nguyên nhân; sai số hiệu chuẩn độ tin cậy (ECE, Brier Score).
Chất lượng quyết định	Tỷ lệ ra quyết định chính xác (Strategy Accuracy); Tỷ lệ tự động hóa mất an toàn (Unsafe Automation Rate); Tỷ lệ tái huấn luyện không cần thiết (Unnecessary Retraining Rate); Tỷ lệ bỏ sót suy giảm (Missed Degradation Rate).
Năng lực phục hồi	Thời gian phục hồi (Time-to-Recovery); Đường cong phục hồi hiệu năng (F1 recovery curve); Mức cải thiện hiệu năng sau CT (Δ_{recovery}).
Hiệu quả chi phí	Tổng số giờ tính toán (GPU/CPU-hours); Tỷ lệ cắt giảm tái huấn luyện thừa (URR); Chi phí phục vụ shadow mode.
Gánh nặng chuyên gia	Số lượng yêu cầu duyệt thủ công (Manual Approval Count); Tỷ lệ can thiệp của con người (Human Intervention Rate); Thời gian phản hồi trung bình.
An toàn triển khai	Tỷ lệ vi phạm guardrails; Tỷ lệ thu hồi tự động thành công (Rollback success); Tỷ lệ candidate bị từ chối tại Evaluation Gate.

Ba chỉ số cốt lõi trực tiếp chứng minh đóng góp khoa học của đề tài:

$$\text{Unnecessary Retraining Rate (URR)} = \frac{\text{Số lần CT khi không có suy giảm có hại}}{\text{Tổng số lần kích hoạt CT}} \quad (1)$$

$$\text{Missed Degradation Rate (MDR)} = \frac{\text{Số ca suy giảm có hại bị bỏ qua không xử lý}}{\text{Tổng số ca suy giảm có hại thực tế}} \quad (2)$$

$$\text{Unnecessary Retraining Reduction} = 1 - \frac{\text{URR}_{\text{Proposed}}}{\text{URR}_{\text{Baseline}}} \quad (3)$$

8.4 Mô hình Phân tích Hiệu quả Đa mục tiêu (Cost-Effectiveness Model)

Để loại bỏ sự thiếu nhất quán về mặt thứ nguyên, hiệu quả của chính sách bảo trì mô hình được lượng hóa qua hàm thỏa dụng tổng hợp (Composite Utility Function):

$$\text{Utility} = \Delta\text{Performance Gained} - \lambda_1 \cdot C_{\text{compute}} - \lambda_2 \cdot C_{\text{human}} - \lambda_3 \cdot \text{Risk}_{\text{unsafe}}$$

trong đó:

- $\Delta\text{Performance Gained}$ là mức độ phục hồi hiệu năng F1 tích lũy theo thời gian.
- C_{compute} là tổng chi phí tài nguyên tính toán (chuẩn hóa theo GPU/CPU-hours).
- C_{human} là chi phí nỗ lực chuyên gia (số lượt can thiệp thủ công $\times$ thời gian xử lý).
- $\text{Risk}_{\text{unsafe}}$ là tổn thất do các quyết định tự động hóa sai lầm gây ra (phạt nặng các trường hợp thăng cấp candidate lỗi).
- $\lambda_1, \lambda_2, \lambda_3 \geq 0$ là các hệ số chuẩn hóa và trọng số đánh đổi theo yêu cầu nghiệp vụ.

8.5 Thiết kế Thử nghiệm Bóc tách và Độ nhạy (Ablation & Sensitivity)

8.5.1 Ablation Study (Đánh giá đóng góp từng thành phần)

Nghiên cứu tiến hành bóc tách 6 cấu hình lũy tiến để chứng minh giá trị của từng khối chức năng:

- A_0 : Kích hoạt CT thuần túy dựa trên Data Drift đơn lẻ.
- A_1 : Bổ sung phát hiện đa tín hiệu (Multi-Signal Detection: Data drift + Quality + Confidence).
- A_2 : Bổ sung Khối chẩn đoán phân loại nguyên nhân (A_1 + Diagnosis).
- A_3 : Bổ sung Thang đo mức độ nghiêm trọng (A_2 + Severity S_0 – S_4).
- A_4 : Bổ sung Chính sách can thiệp thích ứng (A_3 + Adaptive Policy).
- A_5 : Bổ sung Vòng lặp chuyên gia (A_4 + HITL).
- A_6 : Toàn bộ hệ thống hoàn chỉnh (A_5 + EvaluationGate + Shadow Mode).

8.5.2 Sensitivity Analysis (Phân tích độ nhạy tham số)

Kiểm chứng tính bền vững của chính sách khi thay đổi các ngưỡng quyết định:

- Ngưỡng phủ nhận: $r_{\min} \in \{0.3, 0.5, 0.7, 0.9\}$.
- Ngưỡng suy giảm hiệu năng: $\theta_{\text{perf}} \in \{0.03, 0.05, 0.08, 0.10\}$.
- Ngưỡng trôi dạt: $\theta_{\text{drift}} \in \{0.1, 0.2, 0.3\}$ (khoảng cách Wasserstein/PSI).
- Thời gian giãn cách: Cooldown $\in \{12\text{h}, 24\text{h}, 72\text{h}, 168\text{h}\}$.

9 Quản trị rủi ro và Ranh giới an toàn hệ thống

- **Thách thức về nhãn trễ (Delayed Labels):** Hệ thống áp dụng nguyên tắc phân tách ranh giới giữa giám sát tín hiệu đại diện (Proxy Monitoring) và tính toán hiệu năng đã xác thực (Confirmed Performance); chỉ kích hoạt CT khi độ phủ nhận r_{labels} đạt ý nghĩa thống kê (chi tiết tại Mục 5.5).
- **Tính mơ hồ trong quan hệ nhân quả (Causal Ambiguity):** Khi xảy ra đồng thời nhiều dạng biến động, khối chẩn đoán xếp hạng các nguyên nhân theo xác suất và tự động rơi về chế độ an toàn HUMAN_REVIEW, tránh việc suy diễn nguyên nhân tuyệt đối khi thiếu chứng cứ.
- **Ngăn ngừa bão táp huấn luyện (Training Storm):** Thiết lập thời gian giãn cách tối thiểu (Cooldown Period) giữa hai lần CT liên tiếp của cùng một mô hình và đặt ngưỡng trần ngân sách tính toán (Budget Cap) theo tuần/tháng.
- **Bảo mật và cô lập đa tenant (Multi-Tenancy Isolation):** Mọi bản ghi sự kiện, dữ liệu phản hồi và truy vấn bằng chứng đều được gắn khóa phân vùng cứng theo `tenant_id`, tuân thủ chính sách lưu giữ và xóa dữ liệu định kỳ.
- **Kiểm soát sai số tổng quát hóa (External Validity):** Các kịch bản thử nghiệm được tạo với nhiều mức cường độ, kết hợp dữ liệu nhiều và độ trễ nhãn ngẫu nhiên trên nhiều hạt giống (seeds) để bảo đảm kết luận thực nghiệm mang tính khách quan.

10 Tổng quan các công trình liên quan và Phân tích khoảng trống

10.1 Khung kiến trúc MLOps và Mô hình trưởng thành

Nghiên cứu của Amou Najafabadi *et al.* [1] là công trình systematic mapping toàn diện nhất hiện nay, phân tích 93 bài báo khoa học để thiết lập khung kiến trúc MLOps chuẩn mực gồm 39 thành phần, 56 quy trình và 8 biến thể kiến trúc. Kreuzberger *et al.* [2] tổng hợp các nguyên lý vận hành, định nghĩa và phân tầng hạ tầng MLOps trong môi trường doanh nghiệp. Mô hình trưởng thành của Google Cloud [4] định nghĩa 3 cấp độ (ML0–ML2), trong đó Continuous Training nằm ở mức trưởng thành cao nhất ML2. Mô hình của Microsoft [5] và nghiên cứu của Granlund *et al.* [14] nhấn mạnh thách thức trong quản trị vòng đời và phân định trách nhiệm đa tổ chức. Tuy nhiên, các khung kiến trúc này dừng lại ở việc mô tả cấu trúc tĩnh hoặc quy định các trigger cơ học, chưa cung cấp giải pháp cụ thể về chẩn đoán nguyên nhân suy giảm và cơ chế ra quyết định thích ứng.

10.2 Giám sát mô hình và Phát hiện trôi dạt

Về mặt thuật toán học thuật, các cơ chế phát hiện trôi dạt kinh điển trên luồng dữ liệu (data streams) gồm ADWIN [9], DDM [10] và tổng quan toàn diện về concept drift của Lu *et al.* [11]. Ở khía cạnh công cụ công nghiệp, Evidently AI [7] cung cấp khả năng tính toán data drift và kiểm tra chất lượng dữ liệu cơ bản; NannyML [8] tập trung vào phương pháp ước lượng hiệu năng (performance estimation) khi chưa có nhãn. Tuy vậy, các công cụ này hoạt động như các module đo lường độc lập, thiếu một vòng lặp khép kín có thể liên kết trực tiếp bằng chứng giám sát với các chính sách can thiệp an toàn và xử lý nhãn trễ thông qua mã định danh dự đoán (`prediction_id`).

10.3 Hệ thống Continuous Training và Tự thích ứng

Luo *et al.* [12] đề xuất pipeline thử nghiệm thích ứng tự động (CTCD-e) dựa trên các công cụ nguồn mở nhằm tự động hóa quy trình huấn luyện và triển khai khi phát hiện drift. Semola *et al.* [13] giới thiệu Continual Learning as a Service (CLaaS) hỗ trợ cập nhật mô hình gia tăng trên điện toán đám mây. Đa số các hệ thống CT hiện nay đều thiết kế theo cơ chế phản xạ: *khi có cảnh báo drift thì tự động kích hoạt huấn luyện lại*. Cách tiếp cận này bộc lộ hạn chế lớn vì chưa phân biệt được trôi dạt lành tính (Benign Drift) với trôi dạt có hại, dẫn đến lãng phí tài nguyên tính toán hoặc tái huấn luyện sai lầm trên dữ liệu rác.

10.4 Quản trị rủi ro, Tính vững chắc và Vòng lặp chuyên gia (HITL)

Bayram & Ahmed [6] khảo sát toàn diện về tính vững chắc (robustness) và quản trị rủi ro trong MLOps, khẳng định việc tự động hóa hoàn toàn mà thiếu kiểm soát an toàn có thể gây thiệt hại nghiêm trọng cho doanh nghiệp. Các nghiên cứu gần đây bắt đầu nhấn mạnh vai trò của Human-in-the-Loop (HITL) trong việc kiểm chứng dữ liệu bất thường và giải quyết các trường hợp suy giảm hiệu năng không rõ nguyên nhân.

10.5 Bảng đánh giá khoảng trống nghiên cứu (Literature Gap Matrix)

Bảng 16 đối chiếu toàn diện đề tài nghiên cứu này với các công trình tiêu biểu trên y văn thể giới theo 8 tiêu chí chức năng cốt lõi:

Table 16: Bảng đối chiếu và đánh giá khoảng trống nghiên cứu.

Công trình / Framework	C1	C2	C3	C4	C5	C6	C7	Khoảng trống chưa giải quyết (Identified Research Gap)
Amou Najafabadi <i>et al.</i> [1]	~	~	×	×	×	×	×	Khung tham chiếu tổng hợp mang tính mô tả; chưa hiện thực hóa thuật toán chẩn đoán hay chính sách CT thích ứng.
Kreuzberger <i>et al.</i> [2]	✓	~	×	×	×	~	×	Định nghĩa CT ở cấp độ kiến trúc tổng quan; kích hoạt CT dạng phản xạ cơ học, thiếu chẩn đoán nguyên nhân và phân tầng Severity.

Công trình / Framework	C1	C2	C3	C4	C5	C6	C7	Khoảng trống chưa giải quyết (Identified Research Gap)
Luo <i>et al.</i> (CTCD-e) [12]	✓	~	×	×	×	~	~	Tái huấn luyện tự động theo lịch/drift đơn lẻ; thiếu cơ chế phân loại trôi dạt lạnh tính vs. có hại và thiếu vòng lặp HITL.
Semola <i>et al.</i> (CLaaS) [13]	~	~	×	×	×	~	×	Tập trung thuật toán học liên tục (continual learning); giả định nhãn luôn sẵn sàng, thiếu chẩn đoán sự cố dữ liệu.
Evidently AI / NannyML [7, 8]	✓	✓	×	×	×	×	×	Công cụ đo lường và ước lượng trôi dạt độc lập; không có vòng lặp ra quyết định hành động hay điều phối vòng đời CT.
Bayram & Ahmed [6]	~	~	×	~	~	×	~	Khảo sát lý thuyết về tính vững chắc và rủi ro; chưa đề xuất một framework vận hành và bộ benchmark định lượng cụ thể.
ADWIN / DDM / Lu [9, 10, 11]	✓	×	×	×	×	×	×	Thuật toán phát hiện biến động trên luồng dữ liệu thô; hoàn toàn nằm ngoài bối cảnh kiến trúc MLOps và pipeline serving/CT.
Đề tài nghiên cứu (Đề xuất)	✓	✓	✓	✓	✓	✓	✓	Giải quyết trọn vẹn chuỗi khép kín: Phát hiện đa tín hiệu → Chẩn đoán nguyên nhân & Severity → Chính sách can thiệp an toàn (No-op/Data Eng/CT/HITL) → Evaluation Gate → Shadow/Canary.

Quy ước ký hiệu: ✓: Hỗ trợ đầy đủ; ~: Hỗ trợ một phần / mức khái niệm; ×: Chưa hỗ trợ / Khoảng trống nghiên cứu.

Ghi chú các tiêu chí đánh giá (C1–C7):

- **C1 (Multi-Signal Drift Detection):** Phát hiện đa tín hiệu (Data drift $P(X)$, Prediction drift $P(\hat{Y})$, Data quality).
- **C2 (Performance Monitoring):** Giám sát hiệu năng thực tế ($P(Y|X)$, F1, Precision, Recall).
- **C3 (Delayed-Label Handling):** Xử lý nhãn trễ dựa trên `prediction_id` và cửa sổ trượt watermark.
- **C4 (Root-Cause Diagnosis):** Phân loại nguyên nhân suy giảm (7 nhóm) và định lượng mức độ nghiêm trọng (S_0 – S_4).
- **C5 (Adaptive Action Space):** Không gian hành động đa nhánh (No-op, Data Engineering, Incremental CT, Full CT, HITL).
- **C6 (Safe Evaluation Gate):** Cổng thẩm định đối kháng Champion–Candidate đa tiêu chí trước khi triển khai.
- **C7 (Traffic Experiment & Rollback):** Hỗ trợ Shadow Mode, Canary rollout và cơ chế thu hồi tự động khi vi phạm rào chắn.

11 Kết luận và Đóng góp khoa học

Quá trình đối chiếu 39 thành phần kiến trúc và 56 bước quy trình khẳng định repository MLOps PaaS đã sở hữu nền tảng vững chắc về quản trị vòng đời mô hình bất biến, hạ tầng huấn luyện đa dạng và kênh thu thập sự kiện

suy luận. Đóng góp khoa học cốt lõi của đề tài không phải là việc xây dựng thêm một nền tảng MLOps platform hay gắn thêm trigger tái huấn luyện cơ học, mà là đề xuất và kiểm chứng **Khung ra quyết định nhận thức suy giảm (Degradation-Aware Decision Framework)** giúp hệ thống tự thích ứng an toàn và tối ưu chi phí. Trong cấu trúc này, repository MLOps PaaS đóng vai trò là **nền tảng thực nghiệm nguồn mở (Experimental Platform)** phục vụ đánh giá và tái lập kết quả.

Ba đóng góp khoa học chính (Scientific Contributions) bao gồm:

1. **Khung phân loại nguyên nhân và mức độ suy giảm (Degradation Taxonomy & Severity Model):** Hệ thống hóa 7 nhóm nguyên nhân suy giảm và 5 cấp độ nghiêm trọng (S_0-S_4), phân tách rành mạch hiện tượng trôi dạt dữ liệu ($P(X)$) khỏi suy giảm hiệu năng thực tế ($P(Y|X)$) và sự cố chất lượng dữ liệu.
2. **Chính sách can thiệp thích ứng nhận thức rủi ro (Risk-Aware Adaptive Intervention Policy):** Thuật toán chẩn đoán có thể giải thích được và chính sách điều phối hành động đa nhánh (No-op, Data Engineering, Incremental CT, Full CT, HITL), tích hợp cơ chế watermark xử lý nhãn trễ và quản trị ngân sách tính toán.
3. **Giao thức Đánh giá Thực nghiệm Chuẩn hóa (Standardized Benchmark Protocol):** Bộ benchmark toàn diện gồm 12 kịch bản (S_0-S_{11}) có gắn sẵn nguyên nhân suy giảm thực tế (ground_truth_cause), 6 baseline đối chứng (B_0-B_5) và hệ thống chỉ số đo lường hiệu quả cắt giảm CT thừa (URR), ngăn ngừa bỏ sót suy giảm (MDR) và bảo đảm an toàn tự động hóa.

References

- [1] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, and P. Lago, “An architectural perspective on MLOps: Structures, processes, tools, and stakeholders,” *Information and Software Technology*, vol. 193, Art. 108029, 2026. doi: [10.1016/j.infsof.2026.108029](https://doi.org/10.1016/j.infsof.2026.108029).
- [2] P. Kreuzberger, N. Kühn, and S. Hirschl, “Machine learning operations (MLOps): Overview, definition, and architecture,” *IEEE Access*, vol. 11, pp. 31866–31879, 2023. doi: [10.1109/ACCESS.2023.3262138](https://doi.org/10.1109/ACCESS.2023.3262138).
- [3] MLOps PaaS repository, source inspection of control plane, model server, consumer, Evidently, training, registry, deployment, GitOps manifests, and architecture documentation, accessed August 28, 2026.
- [4] K. Salama, J. Kazmierczak, and D. Schut, “Practitioners guide to MLOps: A framework for continuous delivery and automation of machine learning,” Google Cloud, 2021.
- [5] Microsoft, “MLOps maturity model,” Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.
- [6] F. Bayram and B. S. Ahmed, “Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach,” *ACM Computing Surveys*, vol. 57, no. 5, pp. 1–35, 2025. doi: [10.1145/3697080](https://doi.org/10.1145/3697080).
- [7] Evidently AI, “Evidently: Open-source ML monitoring,” <https://www.evidentlyai.com/>.
- [8] NannyML, “NannyML: Performance estimation and monitoring,” <https://www.nannyml.com/>.
- [9] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in *Proc. SDM*, SIAM, 2007, pp. 443–448.
- [10] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *SBIA*, Springer, 2004, pp. 286–295.
- [11] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, 2019.
- [12] Y. Luo, M. Raatikainen, and J. K. Nurminen, “Autonomously adaptive machine learning systems: Experimentation-driven open-source pipeline,” in *Proc. IEEE International Conference on Software Architecture Companion (ICSA-C)*, 2023, pp. 21–28.
- [13] R. Semola, V. Lomonaco, and D. Bacciu, “Continual-learning-as-a-service (CLaaS): On-demand efficient adaptation of predictive models,” *arXiv preprint arXiv:2305.14864*, 2023.
- [14] T. Granlund, A. Kopponen, V. Stirbu, L. Myllyaho, and T. Mikkonen, “MLOps challenges in multi-organization setup: Recent advances and empirical evidence,” in *Proc. WAIN*, IEEE/ACM, 2021, pp. 88–95.